ExecuteSpec Enterprise

Plain-English Designer PRD

Every product concept, journey, Figma flow, and accepted screen explained for a reader who does not know the internal terminology.

Version	1.1 - plain-English revision
Date	2026-07-31
Checkpoint	CP-078
Coverage	110 screens / 14 journeys / 20 named Figma flows
Authority	Product and UX explanation; implementation remains paused

In one sentence

ExecuteSpec Enterprise is a shared workspace that helps a company understand work, decide what should happen, let AI and people carry out only approved actions, prove the result, and keep the full history searchable.

This document describes intended user experience. It does not claim the software, integrations, permissions, or production controls are already implemented.

How to read this document

Start with the glossary if terms such as Objective, evidence, or reconciliation are unfamiliar. The journey section explains complete real-world stories. The flow section explains what each named Figma prototype demonstrates. The final section explains every accepted desktop, tablet, and mobile screen individually.

What this product is - in ordinary language

The product is not just a chatbot that writes code. It is intended to become the place where a company can gather the relevant code, tickets, documents, discussions, service information, and people; turn that context into reviewed work; run or coordinate the work within clear limits; prove what happened; and retain the decisions and evidence for later questions.

The main areas

Home / Needs you	Your work inbox: approvals, blocks, access requests, failures, and other items that need your decision.
Objectives	The company's durable work records. Each Objective keeps the goal, owner, scope, plan, progress, decisions, evidence, and history together.
Intelligence	A permission-aware place to ask questions across approved company information, with citations and honest limitations.
Services	A read-first view of software services, environments, owners, incidents, and operational evidence.
Settings	Administration for people, access, integrations, AI models, data, audit, and billing.
Inspect	A temporary read-only panel for checking source material without losing the current task.
Evidence	Proof such as tests, approvals, source versions, receipts, and external delivery results. It appears in context rather than acting as a separate product.

Plain-English glossary

These terms recur throughout the designs. The explanation is the intended user meaning, not an internal engineering definition.

Objective

One durable piece of work with a goal, owner, scope, current state, decisions, evidence, and history. Think of it as the shared case file for an outcome.

Needs you

The home inbox for decisions or problems that require this user's attention. It is not a generic activity feed.

Inspect

A read-only side panel or mobile route used to look at evidence, source material, or details without leaving the main task.

Intelligence

Permission-aware question answering over company information. It must cite sources and admit when information is missing or inaccessible.

Context bundle

The exact set of code, documents, tickets, discussions, decisions, people, and operational evidence collected for an Objective at a point in time.

Governing source

The source an authorised human has chosen as the rule to follow when two sources disagree.

Plan

The proposed steps, targets, tests, cost, risks, and recovery approach that a person reviews before work starts.

Bounded authority

Permission to perform only the approved action, on the approved target, within the approved cost/time/environment limits.

Checkpoint

A saved safe point from which interrupted work can continue.

Verification

The tests and checks used to determine whether the result actually works. 'The agent finished' is not verification.

Evidence

The test results, source versions, approvals, logs, receipts, and other proof supporting a claim.

Epistemic status

A plain statement of how certain the product is: known, inferred, conflicting, unknown, or unmeasured.

Freshness

How recent the information is and whether a newer source version may exist.

Write-back

An update sent to another system, such as posting a Jira comment or changing a ServiceNow record.

Reconciliation

Checking the external system after a write to confirm what actually happened, especially after a timeout or partial failure.

Connector

The authorised relationship between ExecuteSpec and another service such as GitHub, Jira, Slack, Figma, or PagerDuty.

Redaction

Hiding protected information from both the visible screen and assistive technology when the user lacks permission.

Deep link

A link from another product that points to a specific Enterprise item or action.

Continuity token

A one-time, short-lived handoff reference used to continue the same work on another device without putting protected data in the link.

Model route

The approved AI provider and model selected for a particular kind of work, including its data and spending rules.

Superseded

An older attempt or decision that remains in history but is no longer the current one.

Partial

Some parts succeeded or are known, while other parts failed, are missing, or remain unverified.

Unavailable

The source or service cannot currently be reached. This is different from the user being denied access.

Denied

The system knows the user is not permitted to see or perform something and must not reveal protected details.

The 14 complete user journeys

A journey is a real outcome from the user's point of view. Each entry explains the story first, then preserves the formal product and safety requirements.

EJ01 - Zero-information Enterprise setup

EJ02 - Jira-ready work to scoped Objective

EJ03 - Cross-source context and governing-authority resolution

EJ04 - Plan review, approval, rejection, expiry, and invalidation

EJ05 - Objective work, pause, block, recovery, and supersession

EJ06 - Verification, evidence, delivery, and external-write reconciliation

EJ07 - Scoped Intelligence answer with partial, denied, conflicting, or unavailable evidence

EJ08 - Service state and incident investigation

EJ09 - Connector permission, health, disconnect, export, and purge lifecycle

EJ10 - AI model policy and route recovery

EJ11 - Neutral Studio/Enterprise view choice and secure switching

EJ12 - Authorised and failed external deep links

EJ13 - Tablet Inspect and read-only context retrieval

EJ14 - Mobile approval, recovery, and secure desktop handoff

EJ01 - Zero-information Enterprise setup

In plain English: A company using ExecuteSpec for the first time should see an honest blank workspace, not fake projects or invented history.

Example: A new admin chooses Enterprise, connects Jira or GitHub, reviews what data will be read, and creates the first real piece of work.

Who uses it: First organisation administrator or delegated setup owner
What starts it: The user chooses Enterprise or lands in a zero-information tenant.
What they are trying to achieve: Create a truthful Enterprise starting point without inheriting tenant, people, source, Objective, graph, or memory data.

The story: Neutral product choice -> empty Home -> scoped Ask/setup Brief -> review proposal -> create first Objective/connect source.

Safety rule in ordinary language: Bootstrap remains proposal-only until scope, identity, source permissions, and effect are reviewed. Studio context is not transferred automatically.

If something goes wrong: OAuth denial, empty connector result, unsupported source, permission limitation, and abandoned setup preserve a safe empty tenant and resumable draft.

How the user knows they are done: A reviewed first Objective or connection exists with explicit organisation/team/source scope and an audit receipt.

Open the exact Figma journey entry

EJ02 - Jira-ready work to scoped Objective

In plain English: Work already listed in Jira can become a trackable ExecuteSpec Objective without losing who owns it or which repository and ticket it belongs to.

Example: A developer opens a Jira item, checks the team and repository, then links it to an Objective that everyone can follow.

Who uses it: Developer, engineering manager, product manager, or Objective owner
What starts it: The user opens Home work, an authorised Jira deep link, or creates/links an Objective.
What they are trying to achieve: Open authorised ready work and bind it to the correct Organisation, Team, Objective, project, repository, and source versions.

The story: Authorise link -> Home/Needs you -> choose work -> inspect scope -> link/create Objective -> continue to context.

Safety rule in ordinary language: Source systems remain authoritative. Creating/linking an Objective is a reviewed proposal; duplicate or ambiguous binding cannot auto-resolve.

If something goes wrong: Wrong view, expired/denied link, unavailable Jira, stale source, duplicate Objective, and missing permission route to explicit recovery without disclosing protected identifiers.

How the user knows they are done: A durable Objective is bound to exact source objects and opens with current status, owner, scope, and next lifecycle decision.

Open the exact Figma journey entry

EJ03 - Cross-source context and governing-authority resolution

In plain English: When Jira, documentation, code, and Slack disagree, the product shows the disagreement and lets an authorised person decide which source governs the work.

Example: The README says one API version while Confluence says another. The user compares both and records which one is currently authoritative.

Who uses it: Objective owner, SME, product/engineering lead, or authorised reviewer
What starts it: Context retrieval finds partial, stale, conflicting, denied, unavailable, or no-result evidence.
What they are trying to achieve: Understand code, documents, issues, decisions, discussions, delivery, operations, and people context while preserving source authority and contradictions.

The story: Inspect scope/citations -> compare conflict -> identify governing source -> record versioned authority decision -> refresh context -> continue.

Safety rule in ordinary language: Similarity, recency, or model confidence never chooses organisational truth. Human authority decisions are versioned, auditable, and supersedable.

If something goes wrong: Permission denial redacts protected content; unavailable and no-result remain distinct; stale/partial context lists omissions; source changes invalidate dependent decisions.

How the user knows they are done: Context is accepted for planning with citations, omissions, freshness, governing-source decision, and unresolved uncertainty visible.

Open the exact Figma journey entry

EJ04 - Plan review, approval, rejection, expiry, and invalidation

In plain English: Before the system changes anything, a person can see the proposed work, cost, target, risks, proof required, and recovery plan, then approve or reject exactly that version.

Example: An engineering manager approves a plan for one repository and preview environment; changing the repository or budget requires fresh approval.

Who uses it: Objective owner, approver, security/operations authority, or budget owner
What starts it: A Plan is ready, changed, rejected, expired, invalidated, or requires approval.
What they are trying to achieve: Review a versioned Plan and grant only bounded authority for the exact consequence.

The story: Review tasks/tests/dependencies -> inspect effect/cost/environment/evidence/recovery -> approve, reject, or change scope -> start or return.

Safety rule in ordinary language: Approval binds plan/version, actor, target, environment, source SHA, budget, expiry, preconditions, evidence, cancellation, and recovery. Production remediation stays P2/Conditional.

If something goes wrong: Source/scope/provider/budget/environment changes invalidate prior approval; expiry cannot be bypassed; rejection and scope change preserve the draft and rationale.

How the user knows they are done: An immutable approval/rejection receipt exists, and approved work begins only within its exact bounds.

Open the exact Figma journey entry

EJ05 - Objective work, pause, block, recovery, and supersession

In plain English: Users can tell whether work is running, paused, blocked, recovered, or replaced, and they never lose the last safe checkpoint.

Example: A build stops because test infrastructure is unavailable. The user can request access or resume later from the saved point instead of starting blindly.

Who uses it: Developer, Objective owner, or authorised operator
What starts it: Work starts, pauses, blocks, fails partially, recovers, cancels, or is superseded.
What they are trying to achieve: Track bounded work honestly and preserve durable recovery across interruption or replacement.

The story: Start -> show semantic phase/checkpoint -> pause/cancel or encounter block -> inspect cause -> request access/retry/re-plan -> resume or create successor.

Safety rule in ordinary language: Only actions within the approved bound execute. Pause/cancel preserve the checkpoint; superseded work remains queryable evidence and cannot silently become current.

If something goes wrong: Environment/provider/source failures separate process from outcome; partial work is inspectable; recovery identifies exact checkpoint and replacement SHA/action.

How the user knows they are done: Work finishes or stops in a safe explicit state, with current and superseded attempts correctly related.

Open the exact Figma journey entry

EJ06 - Verification, evidence, delivery, and external-write reconciliation

In plain English: Finishing a task is not treated as proof that it worked. Tests, review evidence, delivery, and updates to external systems are checked separately.

Example: Code and Jira update successfully, but Confluence fails. The screen shows the partial result and retries only the failed update.

Who uses it: Developer, reviewer, Objective owner, CI owner, or delivery approver
What starts it: Verification runs or delivery/write-back is requested.
What they are trying to achieve: Separate process completion from proof, run required verification, and reconcile every external delivery effect.

The story: Review verification plan/impact -> run/inspect outcomes -> open evidence ledger -> review delivery -> execute write-back -> reconcile partial/unknown result.

Safety rule in ordinary language: Required tests cannot be silently removed. Generated tests are proposals until baseline/candidate proof is valid. External retries use authoritative read/idempotency evidence, never blind duplicate writes.

If something goes wrong: Environment blocked, flaky, inconclusive, failed, partial, unmeasured, callback failure, revoked permission, and unknown remote result each have distinct recovery.

How the user knows they are done: Evidence satisfies the delivery boundary and remote/durable state agree, or the Objective remains visibly partial/blocked with preserved payload and next action.

Open the exact Figma journey entry

EJ07 - Scoped Intelligence answer with partial, denied, conflicting, or unavailable evidence

In plain English: A person can ask a question across approved company information and see whether the answer is complete, partial, blocked by permission, conflicting, or unavailable.

Example: A manager asks who owns a service. The answer cites Jira and the repository, while clearly saying Slack could not be searched.

Who uses it: Any authorised Enterprise user
What starts it: The user opens Intelligence or global scoped Ask.
What they are trying to achieve: Ask a read-only question over explicit Objective/team/organisation scope and understand exactly what is known, missing, denied, or conflicting.

The story: Choose scope/permissions -> submit query -> loading -> answer/partial/conflict/denied/unavailable/no-result -> inspect citations or propose bounded follow-up.

Safety rule in ordinary language: Search is read-only. Objective and Inspect query histories are separate. Organisation-wide/cross-team Intelligence is P1 and explicitly labelled.

If something goes wrong: Denied answers expose no protected claim; unavailable is not denial; partial lists omissions; conflict offers compare/authority resolution; greenfield unknown does not fabricate history.

How the user knows they are done: The user receives a cited answer or an honest non-answer with valid next action and no scope leakage.

Open the exact Figma journey entry

EJ08 - Service state and incident investigation

In plain English: Teams can inspect service and incident information without accidentally giving the product broad permission to change production.

Example: During an outage, an SRE reviews current evidence and creates a remediation Objective; the product does not expose a generic production rollback button.

Who uses it: Service owner, SRE, incident commander, engineering manager, or investigator
What starts it: The user opens Services, a service entity, or active incident.
What they are trying to achieve: Understand current service/environment state and investigate incidents without implying unsafe production authority.

The story: Portfolio -> service detail/live evidence -> incident investigation -> proposal/escalation/remediation Objective -> verification -> write-back result.

Safety rule in ordinary language: P0/P1 is read-first. Production rollback/remediation is P2/Conditional; current screens may recommend, request approval, escalate, or create an Objective, not execute generic rollback.

If something goes wrong: Live unavailable uses last observed history with current state unknown; blocked action shows missing preconditions; verified remediation can still have failed ServiceNow/PagerDuty write-back.

How the user knows they are done: Investigation produces a bounded Objective/escalation or verified evidence, with incident record write-back reconciled separately.

Open the exact Figma journey entry

EJ09 - Connector permission, health, disconnect, export, and purge lifecycle

In plain English: Admins can understand what every integration can read or write, whether it is healthy, and what happens when it is disconnected or deleted.

Example: Before connecting Slack, the admin sees the requested channels and permissions. Before purging data, they see what will be retained and why.

Who uses it: Organisation admin, integration admin, security owner, or data owner
What starts it: A connector is added, unhealthy/drifted, re-authorised, disconnected, exported, or purged.
What they are trying to achieve: Connect and operate Jira, Confluence, GitHub, Bitbucket, Slack, Figma, CI, document, incident, and cloud integrations with least privilege and explicit lifecycle effects.

The story: Review capabilities/scopes -> OAuth/validate -> monitor connection/permission/availability/freshness/capability -> repair drift or disconnect -> export -> confirm purge -> verify stores.

Safety rule in ordinary language: Requested permissions, capability, identity, retention, legal hold, affected Objects, irreversible boundary, and recovery are reviewed. Logos may be absent but canonical provider names remain.

If something goes wrong: OAuth denied, missing scope, rate limit, webhook/cursor/API drift, failed write, partial purge, legal hold, and revoked permission have connector-specific recovery.

How the user knows they are done: Connector is validated/healthy, safely disconnected with export, or verified purged with complete audit evidence.

Open the exact Figma journey entry

EJ10 - AI model policy and route recovery

In plain English: Admins and approvers can see which AI provider and model will be used, what it costs, and what alternatives are allowed. The product never swaps models silently.

Example: A model reaches its usage limit. The user reviews an approved alternative and resumes only after the new route is validated.

Who uses it: AI platform admin, governance owner, budget owner, or authorised Objective approver
What starts it: The user reviews AI policy or an Objective call is blocked by route/usage policy.
What they are trying to achieve: Understand exact provider/model routes, data/budget policy, health, usage limits, and explicit alternatives without silent failover.

The story: Inspect matrix -> select allowed route/candidate -> review provider/model/data/budget/failure boundaries -> approve policy/route change -> validate -> resume or remain blocked.

Safety rule in ordinary language: Route changes are high impact. Exact provider and model remain visible; DeepSeek/OpenRouter/Fireworks stay named without logos. No hidden substitution or spend.

If something goes wrong: Unavailable provider, health degradation, quota/usage limit, policy denial, missing key, paused-call uncertainty, and no eligible route each show separate recovery.

How the user knows they are done: An allowed validated route is active with audit receipt, or the action remains explicitly blocked with approved alternatives.

Open the exact Figma journey entry

EJ11 - Neutral Studio/Enterprise view choice and secure switching

In plain English: Studio and Enterprise are separate products. Choosing one must not silently carry personal or company information into the other.

Example: A freelancer who also belongs to a company chooses Enterprise, signs into the company account, and only then sees company Objectives.

Who uses it: User with access to one or both products
What starts it: The user chooses Enterprise/Studio or switches view.
What they are trying to achieve: Choose the correct product without leaking tenant or personal context across product boundaries.

The story: Neutral choice -> authenticate/authorise target -> open matching product URL and breakpoint -> resolve permitted explicit references only.

Safety rule in ordinary language: Studio and Enterprise are separate views/routes/storage/context boundaries. Target authorisation occurs before any protected reference is restored.

If something goes wrong: Wrong account/view, denied target, expired session, or unsupported reference returns to neutral choice with no inherited protected identifiers.

How the user knows they are done: The correct product opens with authorised context; no automatic cross-product transfer occurs.

Open the exact Figma journey entry

EJ12 - Authorised and failed external deep links

In plain English: Links from Jira, Slack, email, or Figma open the exact Enterprise item only after the person, account, product view, permission, and link validity are checked.

Example: An expired Jira link shows a safe explanation and a reissue option rather than revealing the ticket title or company name.

Who uses it: Enterprise user entering from Jira, Confluence, Figma, Slack, PR, CI, email, or incident system
What starts it: The user opens the link.
What they are trying to achieve: Open the exact authorised Object/action while preventing wrong-view, expired, denied, or changed-source disclosure.

The story: Open -> authenticate -> choose correct view/account -> evaluate permission and source version -> open exact Object/action or recovery.

Safety rule in ordinary language: Only authorised state resolves protected identifiers. Sign-in/view switch alone is not authorisation. Links are scoped, expiring, revocable, and source-version aware.

If something goes wrong: Wrong view, expired, denied, revoked, source changed, and unavailable target remain mutually distinct with request-access/reissue/return-safe routes.

How the user knows they are done: Authorised users reach the exact Object/action; others receive a redacted recovery state and audit trail.

Open the exact Figma journey entry

EJ13 - Tablet Inspect and read-only context retrieval

In plain English: On a tablet, a user can inspect evidence in a temporary panel and return to exactly where they were without accidentally editing anything.

Example: A reviewer opens a source citation, reads it in the Inspect panel, closes it, and returns to the same approval control.

Who uses it: Tablet user reviewing an Objective, evidence, decision, or source
What starts it: The user taps an evidence/context/decision link or Inspect control.
What they are trying to achieve: Open contextual Inspect without losing the primary Objective and without turning search into mutation.

The story: Primary surface -> open 560px modal Inspect -> focus trap/read-only Search context -> inspect/copy/open source -> close/Escape/back -> deterministic focus return.

Safety rule in ordinary language: Background is inert while the modal is open. Search context is permission-scoped and read-only; Objective composer state/history stays separate.

If something goes wrong: Unavailable/denied/no-result search remains inside Inspect with redaction; orientation/keyboard changes preserve selected object and query draft.

How the user knows they are done: Inspect closes to the exact invoking control with Objective, selection, draft, and pending decision preserved.

Open the exact Figma journey entry

EJ14 - Mobile approval, recovery, and secure desktop handoff

In plain English: A mobile user can make safe decisions and securely continue desktop-only work using a short-lived, one-time handoff.

Example: A manager reviews a plan on mobile, then opens a one-time desktop handoff. The desktop checks identity, permission, and source version before restoring the context.

Who uses it: Mobile user supervising an Objective or completing a review
What starts it: The user approves/rejects/changes scope, handles a block/write-back failure, or selects Continue on desktop.
What they are trying to achieve: Complete supported decisions/recovery on mobile and securely continue desktop-required work without losing or exposing context.

The story: Review decision -> act or recover -> create one-time handoff -> desktop reauth/reauthorise/source check -> opened -> complete, expire, revoke, or reissue after source change.

Safety rule in ordinary language: The token is one-time, same-user, scoped, expiring, revocable, contains no protected payload, and restores only after destination permission and source-version checks.

If something goes wrong: Blocked work, partial write-back, wrong user, expired/revoked token, source changed, denied destination, and abandoned handoff preserve Objective and valid recovery.

How the user knows they are done: The mobile action completes or an authorised desktop continuation restores the exact safe state; token is consumed/invalidated with receipt.

Open the exact Figma journey entry

The 20 named Figma prototype flows

A Figma flow is a clickable demonstration, not a separate product feature. It proves that the important path and recovery states are connected for a particular device or job.

F01 - Enterprise Desktop - Core Objective Flow

What this means: Shows the main clickable path for this named Enterprise task and its important alternate states.
What the designer should verify: The user can finish, recover, or return without losing scope, authority, evidence, or draft state.

F02 - Enterprise Desktop - View Choice and Recovery

What this means: Shows the safe choice between personal Studio and company Enterprise.
What the designer should verify: The selected product is authorised before any personal or company context is restored.

F03 - Enterprise Mobile - Core Objective Flow

What this means: Shows the main clickable path for this named Enterprise task and its important alternate states.
What the designer should verify: The user can finish, recover, or return without losing scope, authority, evidence, or draft state.

F04 - Enterprise Mobile - Secure Continuity

What this means: Shows how work moves safely between mobile, tablet, and desktop with a one-time handoff.
What the designer should verify: Opened, expired, revoked, source-changed, and completed states are distinct and replay is prevented.

F05 - Enterprise Tablet - Intelligence and Scope Recovery

What this means: Shows question answering and recovery across permitted company context.
What the designer should verify: Answers remain honest about missing, conflicting, stale, denied, or unavailable evidence.

F06 - Enterprise Tablet - Connector Authorization Recovery

What this means: Shows how an admin reviews and authorises an integration, including failure recovery.
What the designer should verify: Requested permissions and effects are understood before data is read, and failed authorization leaves no partial connection.

F07 - Enterprise Tablet - Operations and Governance

What this means: Shows how service and incident information is investigated from a read-first operational surface.
What the designer should verify: The user can inspect, escalate, or create bounded work without receiving generic production mutation authority.

F08 - Enterprise Mobile - Intelligence, Incident and Settings

What this means: Shows how service and incident information is investigated from a read-first operational surface.
What the designer should verify: The user can inspect, escalate, or create bounded work without receiving generic production mutation authority.

F09 - Enterprise Tablet - External Link Authorized

What this means: Shows the successful path from an outside link to the exact authorised Enterprise item.
What the designer should verify: Protected information appears only after account, view, permission, expiry, and source checks pass.

F10 - Enterprise Tablet - Secure Continuity

What this means: Shows how work moves safely between mobile, tablet, and desktop with a one-time handoff.
What the designer should verify: Opened, expired, revoked, source-changed, and completed states are distinct and replay is prevented.

F11 - Enterprise Tablet - External Link Recovery

What this means: Shows the safe alternatives when an outside link is wrong, expired, denied, or otherwise unusable.
What the designer should verify: No protected target information leaks, and the user has a clear request-access, reissue, switch, or return path.

F12 - Enterprise Mobile - External Link Authorized

What this means: Shows the successful path from an outside link to the exact authorised Enterprise item.
What the designer should verify: Protected information appears only after account, view, permission, expiry, and source checks pass.

F13 - Enterprise Mobile - External Link Recovery

What this means: Shows the safe alternatives when an outside link is wrong, expired, denied, or otherwise unusable.
What the designer should verify: No protected target information leaks, and the user has a clear request-access, reissue, switch, or return path.

F14 - Enterprise Desktop - Scoped Ask

What this means: Shows how a user asks a question over an explicit, permission-checked set of company sources.
What the designer should verify: The answer cites sources and clearly handles partial, conflicting, denied, unavailable, and no-result outcomes.

F15 - Enterprise Tablet - Scoped Ask

What this means: Shows how a user asks a question over an explicit, permission-checked set of company sources.
What the designer should verify: The answer cites sources and clearly handles partial, conflicting, denied, unavailable, and no-result outcomes.

F16 - Enterprise Mobile - Scoped Ask

What this means: Shows how a user asks a question over an explicit, permission-checked set of company sources.
What the designer should verify: The answer cites sources and clearly handles partial, conflicting, denied, unavailable, and no-result outcomes.

F17 - Enterprise Desktop - Zero-info Setup

What this means: Shows how a brand-new company moves from an empty workspace to a reviewed first setup or Objective.
What the designer should verify: The user can start safely without fake data, review permissions, and recover from an abandoned or failed setup.

F18 - Enterprise Tablet - Zero-info Setup

What this means: Shows how a brand-new company moves from an empty workspace to a reviewed first setup or Objective.
What the designer should verify: The user can start safely without fake data, review permissions, and recover from an abandoned or failed setup.

F19 - Enterprise Mobile - Zero-info Setup

What this means: Shows how a brand-new company moves from an empty workspace to a reviewed first setup or Objective.
What the designer should verify: The user can start safely without fake data, review permissions, and recover from an abandoned or failed setup.

F20 - Enterprise Mobile - Model Route Recovery

What this means: Shows what happens when the chosen AI provider or model cannot be used.
What the designer should verify: The user sees the reason, policy, cost, and approved alternatives; no silent provider switch occurs.

Every accepted screen, explained

The following 110 entries are the accepted design authority: 48 desktop, 30 tablet, and 32 mobile. Each title opens the exact Figma node. The green box is the non-technical explanation a designer can use when discussing the screen with a customer, founder, or stakeholder.

Desktop screens

EN-D-00A - Neutral view choice

In plain English: This is the safe front door where the user chooses personal Studio or company Enterprise before any protected information appears. On desktop, the same meaning must remain clear.

Why this screen exists: It prevents personal and company context from being mixed.
Example: A user selects Enterprise, authenticates for the company, and then sees only company data they may access.

What the user is trying to do: Choose the correct product boundary.
What must be visible: Neutral Studio/Enterprise choices; no tenant/project/Objective/source/member/memory disclosure; target authorisation and separate URL.
What the user can do: Choose a product or return safely.
When something goes wrong: Wrong account/view or denied target clears protected context and returns to neutral choice.

Design reference: 1440x960 (desktop); 1 canonical actions; 1 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ11

EN-D-01Z - Zero information

In plain English: This is the honest first-use experience for a company that has not connected any source or created any Objective yet. On desktop, the same meaning must remain clear.

Why this screen exists: The product must guide setup without pretending that sample teams, tickets, or history are real.
Example: The admin can describe the first goal, review a setup proposal, or connect a source such as Jira or GitHub.

What the user is trying to do: Bootstrap a truthful empty tenant.
What must be visible: Empty-state explanation; organisation/team/setup intent; source options; proposal-only Brief; permissions/effect review; cancel/resume.
What the user can do: Open scoped Ask, draft setup, review proposal, or connect a source.
When something goes wrong: No sample data may look real; failed setup preserves a safe empty tenant and resumable draft.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ01

EN-D-02 - Zero-info setup brief

In plain English: This is the honest first-use experience for a company that has not connected any source or created any Objective yet. On desktop, the same meaning must remain clear.

Why this screen exists: The product must guide setup without pretending that sample teams, tickets, or history are real.
Example: The admin can describe the first goal, review a setup proposal, or connect a source such as Jira or GitHub.

What the user is trying to do: Bootstrap a truthful empty tenant.
What must be visible: Empty-state explanation; organisation/team/setup intent; source options; proposal-only Brief; permissions/effect review; cancel/resume.
What the user can do: Open scoped Ask, draft setup, review proposal, or connect a source.
When something goes wrong: No sample data may look real; failed setup preserves a safe empty tenant and resumable draft.

Design reference: 1440x960 (desktop); 3 canonical actions; 3 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ01

EN-D-05A - Scoped Ask

In plain English: This lets the user ask a question while explicitly choosing which team, Objective, and sources may be searched. On desktop, the same meaning must remain clear.

Why this screen exists: Users need to understand the boundary of the answer before trusting it.
Example: A user asks who owns a service and limits the search to the current Objective and its approved sources.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 1440x960 (desktop); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ07

EN-D-10H - Home needs you

In plain English: This is the user's Enterprise inbox. It highlights Objectives that need a decision, access, approval, or recovery action. On desktop, the same meaning must remain clear.

Why this screen exists: People should immediately know what needs them, rather than search through a generic dashboard.
Example: A manager sees one plan awaiting approval and one blocked Objective, then opens the most urgent item.

What the user is trying to do: Find the next Objective decision.
What must be visible: Organisation/team; Needs you; ready/running/blocked Objective rows; source/approval/access causes; filters; maturity; proposal-only create/link.
What the user can do: Open/filter work, create/link Objective, reconnect source, or resolve required authority.
When something goes wrong: Empty, source unavailable, permission-limited, loading, and no-ready-work remain distinct.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ02

EN-D-11U - Home source unavailable

In plain English: This screen says that a connected source cannot currently be reached, while keeping the last trustworthy information clearly separate. On desktop, the same meaning must remain clear.

Why this screen exists: An outage must not look like missing data or a permission denial.
Example: Jira is temporarily down; the user can see when it was last read and choose retry, reconnect, or continue with clearly marked cached evidence.

What the user is trying to do: Find the next Objective decision.
What must be visible: Organisation/team; Needs you; ready/running/blocked Objective rows; source/approval/access causes; filters; maturity; proposal-only create/link.
What the user can do: Open/filter work, create/link Objective, reconnect source, or resolve required authority.
When something goes wrong: Empty, source unavailable, permission-limited, loading, and no-ready-work remain distinct.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ02

EN-D-14D - Context permission denied

In plain English: This is a safe permission-denied state. It explains that access is missing without revealing the protected item. On desktop, the same meaning must remain clear.

Why this screen exists: Even headings, identifiers, and accessibility labels can leak confidential information.
Example: The user can request access or return safely, but cannot see the ticket title or company context.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ03

EN-D-14E - Context searched no result

In plain English: This screen says that the approved search completed but found nothing matching the question. On desktop, the same meaning must remain clear.

Why this screen exists: Finding nothing is different from being denied access or from a source being unavailable, and the product must not invent an answer.
Example: The user changes the wording or expands the approved scope instead of receiving a fabricated result.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ03

EN-D-20C - Objective context conflict

In plain English: This screen appears when two approved sources disagree and shows both claims side by side. On desktop, the same meaning must remain clear.

Why this screen exists: The AI must not quietly choose organisational truth based on confidence or recency alone.
Example: Confluence and the repository name different API owners, so an authorised lead selects the governing source.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ03

EN-D-20R - Context authority resolved

In plain English: This confirms that an authorised person has chosen which source should govern the work. On desktop, the same meaning must remain clear.

Why this screen exists: The decision becomes part of the history and can be replaced later if the source changes.
Example: The team chooses the repository contract over an older document, and planning continues using that version.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 1440x960 (desktop); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ03

EN-D-21P - Context partial stale

In plain English: This screen shows a useful but incomplete result and names exactly what could not be checked. On desktop, the same meaning must remain clear.

Why this screen exists: A partial answer must never look complete.
Example: The answer cites Jira and code, but says Slack was unavailable and one document is stale.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 1440x960 (desktop); 5 canonical actions; 5 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ03

EN-D-23A - Plan approval

In plain English: This is the review screen shown before work starts. It explains what will change, where, how much it may cost, how it will be checked, and how to recover. On desktop, the same meaning must remain clear.

Why this screen exists: Approval must apply to one exact plan and scope, not to vague future activity.
Example: An approver checks the team, repository, branch, environment, cost, and evidence requirements before allowing the plan to proceed.

What the user is trying to do: Review bounded Plan authority.
What must be visible: Plan/version; tasks/tests/dependencies; target/SHA/environment; actor/approver; cost/time/resource caps; expiry; evidence; effect; recovery.
What the user can do: Approve, reject, change scope, inspect proof, revoke, or re-review.
When something goes wrong: Rejected/expired/invalidated states cannot execute; material changes require fresh approval.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ04

EN-D-23B - Plan approved bounded authority

In plain English: This is the receipt showing exactly what was approved and the limits the work must obey. On desktop, the same meaning must remain clear.

Why this screen exists: Users need proof that execution has not gained broader authority than they granted.
Example: The receipt records the plan version, repository, environment, budget, expiry, and approver.

What the user is trying to do: Review bounded Plan authority.
What must be visible: Plan/version; tasks/tests/dependencies; target/SHA/environment; actor/approver; cost/time/resource caps; expiry; evidence; effect; recovery.
What the user can do: Approve, reject, change scope, inspect proof, revoke, or re-review.
When something goes wrong: Rejected/expired/invalidated states cannot execute; material changes require fresh approval.

Design reference: 1440x960 (desktop); 5 canonical actions; 5 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ04

EN-D-23X - Plan rejected expired invalidated

In plain English: This screen explains why a plan cannot run and preserves the proposal for correction or review. On desktop, the same meaning must remain clear.

Why this screen exists: Rejected, expired, or changed plans must never execute using old approval.
Example: The source branch changed after approval, so the user reviews the new plan rather than silently reusing the old decision.

What the user is trying to do: Review bounded Plan authority.
What must be visible: Plan/version; tasks/tests/dependencies; target/SHA/environment; actor/approver; cost/time/resource caps; expiry; evidence; effect; recovery.
What the user can do: Approve, reject, change scope, inspect proof, revoke, or re-review.
When something goes wrong: Rejected/expired/invalidated states cannot execute; material changes require fresh approval.

Design reference: 1440x960 (desktop); 2 canonical actions; 1 wired; 1 explicitly inert.; 1 direct destinations.
Related journeys: EJ04

EN-D-24B - Working blocked

In plain English: This shows that work cannot currently continue, why it stopped, and the last safe checkpoint. On desktop, the same meaning must remain clear.

Why this screen exists: A blocked process is not a failed product outcome, and users need a safe next move.
Example: Tests cannot reach the preview environment; the user requests access or pauses without losing completed work.

What the user is trying to do: Track current Objective execution.
What must be visible: Semantic phase; route/provider; source SHA; budget; durable checkpoint; live verification; blockers; partial/superseded attempts; pause/cancel/recovery.
What the user can do: Inspect, pause, cancel-preserve, request access, retry, re-plan, or open successor.
When something goes wrong: Process/outcome stay separate; recovery resumes exact checkpoint and superseded work remains evidence.

Design reference: 1440x960 (desktop); 5 canonical actions; 4 wired; 1 explicitly inert.; 4 direct destinations.
Related journeys: EJ05

EN-D-24R - Working recovered superseded

In plain English: This confirms that a blocked attempt resumed safely or was replaced by a new current attempt. On desktop, the same meaning must remain clear.

Why this screen exists: The old attempt remains visible so history is not rewritten.
Example: A new source version resumes from the saved checkpoint while the original attempt is marked superseded.

What the user is trying to do: Track current Objective execution.
What must be visible: Semantic phase; route/provider; source SHA; budget; durable checkpoint; live verification; blockers; partial/superseded attempts; pause/cancel/recovery.
What the user can do: Inspect, pause, cancel-preserve, request access, retry, re-plan, or open successor.
When something goes wrong: Process/outcome stay separate; recovery resumes exact checkpoint and superseded work remains evidence.

Design reference: 1440x960 (desktop); 5 canonical actions; 5 wired; 0 explicitly inert.; 5 direct destinations.
Related journeys: EJ05

EN-D-24W - Working objective

In plain English: This is the live progress view for an Objective that is currently being worked on. On desktop, the same meaning must remain clear.

Why this screen exists: Users need meaningful phases, proof, budget, and pause/cancel controls - not a stream of internal agent logs.
Example: A developer sees the current task, files affected, test state, remaining budget, and saved checkpoint.

What the user is trying to do: Track current Objective execution.
What must be visible: Semantic phase; route/provider; source SHA; budget; durable checkpoint; live verification; blockers; partial/superseded attempts; pause/cancel/recovery.
What the user can do: Inspect, pause, cancel-preserve, request access, retry, re-plan, or open successor.
When something goes wrong: Process/outcome stay separate; recovery resumes exact checkpoint and superseded work remains evidence.

Design reference: 1440x960 (desktop); 4 canonical actions; 3 wired; 1 explicitly inert.; 3 direct destinations.
Related journeys: EJ05

EN-D-25E - Evidence and delivery

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On desktop, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ06

EN-D-25F - Verification failed incomplete

In plain English: This explains that the work produced a result but the required checks did not pass. On desktop, the same meaning must remain clear.

Why this screen exists: Completion without proof must not be presented as success.
Example: The build succeeds but an integration test fails, so delivery remains blocked and the failed assertion is shown.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 1440x960 (desktop); 1 canonical actions; 1 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ06

EN-D-25W - Delivery partial write-back

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On desktop, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 1440x960 (desktop); 5 canonical actions; 5 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ06

EN-D-26D - Answer denied

In plain English: This is a safe permission-denied state. It explains that access is missing without revealing the protected item. On desktop, the same meaning must remain clear.

Why this screen exists: Even headings, identifiers, and accessibility labels can leak confidential information.
Example: The user can request access or return safely, but cannot see the ticket title or company context.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ07

EN-D-26P - Answer partial stale conflicting

In plain English: This screen shows a useful but incomplete result and names exactly what could not be checked. On desktop, the same meaning must remain clear.

Why this screen exists: A partial answer must never look complete.
Example: The answer cites Jira and code, but says Slack was unavailable and one document is stale.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 1440x960 (desktop); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ07

EN-D-26U - Answer unavailable

In plain English: This is the question-and-answer area for approved company context, including honest states when sources are missing, conflicting, or unreachable. On desktop, the same meaning must remain clear.

Why this screen exists: The answer should be useful without pretending to know more than the evidence supports.
Example: A user asks about planned work and receives citations plus a clear note that one source is unavailable.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ07

EN-D-30Q - Intelligence scoped query

In plain English: This is the question-and-answer area for approved company context, including honest states when sources are missing, conflicting, or unreachable. On desktop, the same meaning must remain clear.

Why this screen exists: The answer should be useful without pretending to know more than the evidence supports.
Example: A user asks about planned work and receives citations plus a clear note that one source is unavailable.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ07

EN-D-38I - Incident conditional remediation

In plain English: This is the investigation workspace for an active service incident. On desktop, the same meaning must remain clear.

Why this screen exists: Teams need current evidence, owners, impact, and safe next actions in one place.
Example: An SRE reviews live and historical signals, then creates a bounded remediation Objective or escalates.

What the user is trying to do: Understand services and investigate incidents.
What must be visible: Service/owner/environment/readiness/incident/change; observation time; live/history distinction; omissions; remediation maturity; verification/write-back.
What the user can do: Open service, inspect evidence/runbook, investigate, create remediation Objective, escalate, or reconcile incident write-back.
When something goes wrong: Live unavailable means current unknown; P0/P1 does not expose executable production rollback.

Design reference: 1440x960 (desktop); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ08

EN-D-39H - Connector drift

In plain English: This screen explains an integration's permissions, health, data use, and lifecycle action in ordinary language. On desktop, the same meaning must remain clear.

Why this screen exists: Connecting, disconnecting, or deleting data can have broad consequences and must be reviewed explicitly.
Example: Before authorising Slack, the admin sees the requested access; if authorisation fails, no channels are read.

What the user is trying to do: Review connector capability and lifecycle.
What must be visible: Provider name; account; scopes/capabilities; connection/permission/availability/freshness/capability dimensions; retention/legal hold; export/purge effect.
What the user can do: Authorise/validate, narrow/rotate, reconnect, export, disconnect, queue/verify purge, or cancel.
When something goes wrong: OAuth denial, drift, rate limit, failed write, partial purge, and legal hold have separate recovery; secrets remain write-only.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ09

EN-D-39P - Disconnect purge export

In plain English: This screen explains an integration's permissions, health, data use, and lifecycle action in ordinary language. On desktop, the same meaning must remain clear.

Why this screen exists: Connecting, disconnecting, or deleting data can have broad consequences and must be reviewed explicitly.
Example: Before authorising Slack, the admin sees the requested access; if authorisation fails, no channels are read.

What the user is trying to do: Review connector capability and lifecycle.
What must be visible: Provider name; account; scopes/capabilities; connection/permission/availability/freshness/capability dimensions; retention/legal hold; export/purge effect.
What the user can do: Authorise/validate, narrow/rotate, reconnect, export, disconnect, queue/verify purge, or cancel.
When something goes wrong: OAuth denial, drift, rate limit, failed write, partial purge, and legal hold have separate recovery; secrets remain write-only.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ09

EN-D-40P - Services portfolio

In plain English: This is the high-level list of services, owners, environments, health observations, and active incidents. On desktop, the same meaning must remain clear.

Why this screen exists: It helps teams find operational context without implying that ExecuteSpec can freely change production.
Example: An engineering manager sees which services have stale health data or an active incident and opens one for investigation.

What the user is trying to do: Understand services and investigate incidents.
What must be visible: Service/owner/environment/readiness/incident/change; observation time; live/history distinction; omissions; remediation maturity; verification/write-back.
What the user can do: Open service, inspect evidence/runbook, investigate, create remediation Objective, escalate, or reconcile incident write-back.
When something goes wrong: Live unavailable means current unknown; P0/P1 does not expose executable production rollback.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ08

EN-D-41U - Service live unavailable

In plain English: This screen says current live service data is unavailable and labels older information as history only. On desktop, the same meaning must remain clear.

Why this screen exists: Old observations must not be mistaken for current health.
Example: The last successful check was two hours ago, so the present state is shown as unknown.

What the user is trying to do: Understand services and investigate incidents.
What must be visible: Service/owner/environment/readiness/incident/change; observation time; live/history distinction; omissions; remediation maturity; verification/write-back.
What the user can do: Open service, inspect evidence/runbook, investigate, create remediation Objective, escalate, or reconcile incident write-back.
When something goes wrong: Live unavailable means current unknown; P0/P1 does not expose executable production rollback.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ08

EN-D-42B - Incident action blocked

In plain English: This shows that a potentially dangerous incident action is not allowed because required authority or safety conditions are missing. On desktop, the same meaning must remain clear.

Why this screen exists: The product must support investigation without creating a generic production-control surface.
Example: The user can inspect evidence, open a runbook, or escalate, but cannot trigger production rollback.

What the user is trying to do: Understand services and investigate incidents.
What must be visible: Service/owner/environment/readiness/incident/change; observation time; live/history distinction; omissions; remediation maturity; verification/write-back.
What the user can do: Open service, inspect evidence/runbook, investigate, create remediation Objective, escalate, or reconcile incident write-back.
When something goes wrong: Live unavailable means current unknown; P0/P1 does not expose executable production rollback.

Design reference: 1440x960 (desktop); 4 canonical actions; 3 wired; 1 explicitly inert.; 2 direct destinations.
Related journeys: EJ08

EN-D-42I - Active incident investigation

In plain English: This is the investigation workspace for an active service incident. On desktop, the same meaning must remain clear.

Why this screen exists: Teams need current evidence, owners, impact, and safe next actions in one place.
Example: An SRE reviews live and historical signals, then creates a bounded remediation Objective or escalates.

What the user is trying to do: Understand services and investigate incidents.
What must be visible: Service/owner/environment/readiness/incident/change; observation time; live/history distinction; omissions; remediation maturity; verification/write-back.
What the user can do: Open service, inspect evidence/runbook, investigate, create remediation Objective, escalate, or reconcile incident write-back.
When something goes wrong: Live unavailable means current unknown; P0/P1 does not expose executable production rollback.

Design reference: 1440x960 (desktop); 6 canonical actions; 6 wired; 0 explicitly inert.; 5 direct destinations.
Related journeys: EJ08

EN-D-42V - Incident verified write-back failed

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On desktop, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 1440x960 (desktop); 4 canonical actions; 3 wired; 1 explicitly inert.; 3 direct destinations.
Related journeys: EJ08

EN-D-50H - Enterprise Settings home

In plain English: This is an administration screen for integrations, access, AI policy, data handling, audit, or billing. On desktop, the same meaning must remain clear.

Why this screen exists: High-impact company settings need clear role, effect, review, and recovery information.
Example: An admin reviews a change, sees who and what it affects, then applies or discards it.

What the user is trying to do: Find the next Objective decision.
What must be visible: Organisation/team; Needs you; ready/running/blocked Objective rows; source/approval/access causes; filters; maturity; proposal-only create/link.
What the user can do: Open/filter work, create/link Objective, reconnect source, or resolve required authority.
When something goes wrong: Empty, source unavailable, permission-limited, loading, and no-ready-work remain distinct.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ02

EN-D-53C - First connector permission impact

In plain English: This screen explains an integration's permissions, health, data use, and lifecycle action in ordinary language. On desktop, the same meaning must remain clear.

Why this screen exists: Connecting, disconnecting, or deleting data can have broad consequences and must be reviewed explicitly.
Example: Before authorising Slack, the admin sees the requested access; if authorisation fails, no channels are read.

What the user is trying to do: Review connector capability and lifecycle.
What must be visible: Provider name; account; scopes/capabilities; connection/permission/availability/freshness/capability dimensions; retention/legal hold; export/purge effect.
What the user can do: Authorise/validate, narrow/rotate, reconnect, export, disconnect, queue/verify purge, or cancel.
When something goes wrong: OAuth denial, drift, rate limit, failed write, partial purge, and legal hold have separate recovery; secrets remain write-only.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ09

EN-D-55M - Route unavailable usage limit

In plain English: This screen explains which AI provider and model will be used, whether it is available, what it may cost, and which alternatives are allowed. On desktop, the same meaning must remain clear.

Why this screen exists: The system must not silently switch providers, change data handling, or spend more money.
Example: The selected model reaches its limit, so the user reviews an approved alternative before resuming.

What the user is trying to do: Govern provider/model routing.
What must be visible: Provider/model names; key source; health; policy/maturity; data/budget boundary; usage/limit; eligible alternatives; approval and route-validation receipt.
What the user can do: Select/review allowed route, request policy change, validate, resume, or remain blocked.
When something goes wrong: No silent failover/substitution/spend; DeepSeek/OpenRouter/Fireworks remain named without logos.

Design reference: 1440x960 (desktop); 3 canonical actions; 2 wired; 1 explicitly inert.; 2 direct destinations.
Related journeys: EJ10

EN-D-55P - AI model policy matrix

In plain English: This screen explains which AI provider and model will be used, whether it is available, what it may cost, and which alternatives are allowed. On desktop, the same meaning must remain clear.

Why this screen exists: The system must not silently switch providers, change data handling, or spend more money.
Example: The selected model reaches its limit, so the user reviews an approved alternative before resuming.

What the user is trying to do: Govern provider/model routing.
What must be visible: Provider/model names; key source; health; policy/maturity; data/budget boundary; usage/limit; eligible alternatives; approval and route-validation receipt.
What the user can do: Select/review allowed route, request policy change, validate, resume, or remain blocked.
When something goes wrong: No silent failover/substitution/spend; DeepSeek/OpenRouter/Fireworks remain named without logos.

Design reference: 1440x960 (desktop); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ10

EN-D-59Q - Purge queued

In plain English: This screen explains an integration's permissions, health, data use, and lifecycle action in ordinary language. On desktop, the same meaning must remain clear.

Why this screen exists: Connecting, disconnecting, or deleting data can have broad consequences and must be reviewed explicitly.
Example: Before authorising Slack, the admin sees the requested access; if authorisation fails, no channels are read.

What the user is trying to do: Review connector capability and lifecycle.
What must be visible: Provider name; account; scopes/capabilities; connection/permission/availability/freshness/capability dimensions; retention/legal hold; export/purge effect.
What the user can do: Authorise/validate, narrow/rotate, reconnect, export, disconnect, queue/verify purge, or cancel.
When something goes wrong: OAuth denial, drift, rate limit, failed write, partial purge, and legal hold have separate recovery; secrets remain write-only.

Design reference: 1440x960 (desktop); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ09

EN-D-60A - External link authorized

In plain English: This confirms that an incoming link is valid and the user is allowed to open the exact Enterprise item. On desktop, the same meaning must remain clear.

Why this screen exists: Protected details appear only after identity, product view, permission, expiry, and source checks pass.
Example: A Jira link opens the intended Objective and action after authorization.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 1440x960 (desktop); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ12

EN-D-60D - Link authorization denied

In plain English: This is a safe permission-denied state. It explains that access is missing without revealing the protected item. On desktop, the same meaning must remain clear.

Why this screen exists: Even headings, identifiers, and accessibility labels can leak confidential information.
Example: The user can request access or return safely, but cannot see the ticket title or company context.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 1440x960 (desktop); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ12

EN-D-60E - Continuity expired

In plain English: This says the cross-device handoff expired before use. On desktop, the same meaning must remain clear.

Why this screen exists: An expired token must not restore the old context.
Example: The user returns to mobile and creates a new handoff if the work is still needed.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 1440x960 (desktop); 1 canonical actions; 1 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ14

EN-D-60W - Wrong view recovery

In plain English: This says the link belongs to a different product view or account and offers a safe way to switch. On desktop, the same meaning must remain clear.

Why this screen exists: The screen must not reveal the protected target before the correct view is authorised.
Example: A Studio session receives an Enterprise link and is asked to switch without seeing the company ticket title.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 1440x960 (desktop); 1 canonical actions; 1 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ12

EN-D-61C - Secure handoff created

In plain English: This confirms that a one-time cross-device handoff has been created. On desktop, the same meaning must remain clear.

Why this screen exists: The link contains only a short-lived reference, not the protected Objective or evidence itself.
Example: A mobile user creates a desktop handoff that expires soon and can be revoked.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ14

EN-D-61D - Continuity completed

In plain English: This confirms that the handoff opened successfully and the one-time token has been consumed. On desktop, the same meaning must remain clear.

Why this screen exists: A used token cannot be replayed.
Example: The desktop restores the authorised Objective once, then invalidates the link.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ14

EN-D-61O - Continuity opened

In plain English: This is the checking stage after a handoff link is opened; it does not restore protected context yet. On desktop, the same meaning must remain clear.

Why this screen exists: The destination must re-check the user, permission, and source version.
Example: The desktop verifies the same user and current source before showing the Objective.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ14

EN-D-61R - Continuity revoked

In plain English: This confirms that the user or system cancelled the handoff and it can no longer be used. On desktop, the same meaning must remain clear.

Why this screen exists: Revocation must be immediate and visible.
Example: A user revokes a lost-device handoff and the old link becomes unusable.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 1440x960 (desktop); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ14

EN-D-61S - Continuity source changed

In plain English: This blocks restoration because the underlying source changed after the handoff was created. On desktop, the same meaning must remain clear.

Why this screen exists: The user must not unknowingly act on stale code, tickets, or evidence.
Example: The repository SHA changed, so the user reviews the difference and creates fresh authority.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 1440x960 (desktop); 5 canonical actions; 4 wired; 1 explicitly inert.; 4 direct destinations.
Related journeys: EJ14

EN-D-62C - External write authoritative read

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On desktop, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 1440x960 (desktop); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ06

EN-D-62R - External write reconciliation

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On desktop, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 1440x960 (desktop); 3 canonical actions; 2 wired; 1 explicitly inert.; 2 direct destinations.
Related journeys: EJ06

Tablet screens

EN-T-00A - Neutral view choice

In plain English: This is the safe front door where the user chooses personal Studio or company Enterprise before any protected information appears. On tablet, the same meaning must remain clear.

Why this screen exists: It prevents personal and company context from being mixed.
Example: A user selects Enterprise, authenticates for the company, and then sees only company data they may access.

What the user is trying to do: Choose the correct product boundary.
What must be visible: Neutral Studio/Enterprise choices; no tenant/project/Objective/source/member/memory disclosure; target authorisation and separate URL.
What the user can do: Choose a product or return safely.
When something goes wrong: Wrong account/view or denied target clears protected context and returns to neutral choice.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ11

EN-T-01 - Home needs you

In plain English: This is the user's Enterprise inbox. It highlights Objectives that need a decision, access, approval, or recovery action. On tablet, the same meaning must remain clear.

Why this screen exists: People should immediately know what needs them, rather than search through a generic dashboard.
Example: A manager sees one plan awaiting approval and one blocked Objective, then opens the most urgent item.

What the user is trying to do: Find the next Objective decision.
What must be visible: Organisation/team; Needs you; ready/running/blocked Objective rows; source/approval/access causes; filters; maturity; proposal-only create/link.
What the user can do: Open/filter work, create/link Objective, reconnect source, or resolve required authority.
When something goes wrong: Empty, source unavailable, permission-limited, loading, and no-ready-work remain distinct.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ02

EN-T-01F - Objective filters

In plain English: This lets the user narrow the Objective list by team, owner, status, urgency, or other useful criteria. On tablet, the same meaning must remain clear.

Why this screen exists: A large company may have hundreds of active Objectives, so people need to find the work relevant to them without changing or hiding the underlying records.
Example: A manager filters to blocked Objectives owned by the payments team, then clears the filter to return to the full list.

What the user is trying to do: Find the next Objective decision.
What must be visible: Organisation/team; Needs you; ready/running/blocked Objective rows; source/approval/access causes; filters; maturity; proposal-only create/link.
What the user can do: Open/filter work, create/link Objective, reconnect source, or resolve required authority.
When something goes wrong: Empty, source unavailable, permission-limited, loading, and no-ready-work remain distinct.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ02

EN-T-02 - Objective context

In plain English: This is the evidence-and-scope view for one Objective. It shows what information the product is using and where that information came from. On tablet, the same meaning must remain clear.

Why this screen exists: A user should be able to judge whether the current plan is based on the right repository, tickets, documents, discussions, and source versions.
Example: Before approving a plan, a lead checks that the correct Jira ticket, repository branch, and current architecture document are included.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ03

EN-T-02Z - Zero-info setup brief

In plain English: This is the honest first-use experience for a company that has not connected any source or created any Objective yet. On tablet, the same meaning must remain clear.

Why this screen exists: The product must guide setup without pretending that sample teams, tickets, or history are real.
Example: The admin can describe the first goal, review a setup proposal, or connect a source such as Jira or GitHub.

What the user is trying to do: Bootstrap a truthful empty tenant.
What must be visible: Empty-state explanation; organisation/team/setup intent; source options; proposal-only Brief; permissions/effect review; cancel/resume.
What the user can do: Open scoped Ask, draft setup, review proposal, or connect a source.
When something goes wrong: No sample data may look real; failed setup preserves a safe empty tenant and resumable draft.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ01

EN-T-03 - Plan decision

In plain English: This is the review screen shown before work starts. It explains what will change, where, how much it may cost, how it will be checked, and how to recover. On tablet, the same meaning must remain clear.

Why this screen exists: Approval must apply to one exact plan and scope, not to vague future activity.
Example: An approver checks the team, repository, branch, environment, cost, and evidence requirements before allowing the plan to proceed.

What the user is trying to do: Review bounded Plan authority.
What must be visible: Plan/version; tasks/tests/dependencies; target/SHA/environment; actor/approver; cost/time/resource caps; expiry; evidence; effect; recovery.
What the user can do: Approve, reject, change scope, inspect proof, revoke, or re-review.
When something goes wrong: Rejected/expired/invalidated states cannot execute; material changes require fresh approval.

Design reference: 834x960 (tablet); 5 canonical actions; 5 wired; 0 explicitly inert.; 5 direct destinations.
Related journeys: EJ04

EN-T-04 - Working objective

In plain English: This is the live progress view for an Objective that is currently being worked on. On tablet, the same meaning must remain clear.

Why this screen exists: Users need meaningful phases, proof, budget, and pause/cancel controls - not a stream of internal agent logs.
Example: A developer sees the current task, files affected, test state, remaining budget, and saved checkpoint.

What the user is trying to do: Track current Objective execution.
What must be visible: Semantic phase; route/provider; source SHA; budget; durable checkpoint; live verification; blockers; partial/superseded attempts; pause/cancel/recovery.
What the user can do: Inspect, pause, cancel-preserve, request access, retry, re-plan, or open successor.
When something goes wrong: Process/outcome stay separate; recovery resumes exact checkpoint and superseded work remains evidence.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ05

EN-T-04B - Working blocked recovery

In plain English: This shows that work cannot currently continue, why it stopped, and the last safe checkpoint. On tablet, the same meaning must remain clear.

Why this screen exists: A blocked process is not a failed product outcome, and users need a safe next move.
Example: Tests cannot reach the preview environment; the user requests access or pauses without losing completed work.

What the user is trying to do: Track current Objective execution.
What must be visible: Semantic phase; route/provider; source SHA; budget; durable checkpoint; live verification; blockers; partial/superseded attempts; pause/cancel/recovery.
What the user can do: Inspect, pause, cancel-preserve, request access, retry, re-plan, or open successor.
When something goes wrong: Process/outcome stay separate; recovery resumes exact checkpoint and superseded work remains evidence.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ05

EN-T-05 - Evidence and delivery

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On tablet, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ06

EN-T-05A - Scoped Ask

In plain English: This lets the user ask a question while explicitly choosing which team, Objective, and sources may be searched. On tablet, the same meaning must remain clear.

Why this screen exists: Users need to understand the boundary of the answer before trusting it.
Example: A user asks who owns a service and limits the search to the current Objective and its approved sources.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ07

EN-T-05P - Partial write-back recovery

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On tablet, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ13

EN-T-06 - Intelligence scoped partial

In plain English: This is the question-and-answer area for approved company context, including honest states when sources are missing, conflicting, or unreachable. On tablet, the same meaning must remain clear.

Why this screen exists: The answer should be useful without pretending to know more than the evidence supports.
Example: A user asks about planned work and receives citations plus a clear note that one source is unavailable.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ07

EN-T-06D - Intelligence permission denied

In plain English: This is the question-and-answer area for approved company context, including honest states when sources are missing, conflicting, or unreachable. On tablet, the same meaning must remain clear.

Why this screen exists: The answer should be useful without pretending to know more than the evidence supports.
Example: A user asks about planned work and receives citations plus a clear note that one source is unavailable.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ07

EN-T-07 - Services portfolio

In plain English: This is the high-level list of services, owners, environments, health observations, and active incidents. On tablet, the same meaning must remain clear.

Why this screen exists: It helps teams find operational context without implying that ExecuteSpec can freely change production.
Example: An engineering manager sees which services have stale health data or an active incident and opens one for investigation.

What the user is trying to do: Understand services and investigate incidents.
What must be visible: Service/owner/environment/readiness/incident/change; observation time; live/history distinction; omissions; remediation maturity; verification/write-back.
What the user can do: Open service, inspect evidence/runbook, investigate, create remediation Objective, escalate, or reconcile incident write-back.
When something goes wrong: Live unavailable means current unknown; P0/P1 does not expose executable production rollback.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ08

EN-T-08 - Incident conditional remediation

In plain English: This is the investigation workspace for an active service incident. On tablet, the same meaning must remain clear.

Why this screen exists: Teams need current evidence, owners, impact, and safe next actions in one place.
Example: An SRE reviews live and historical signals, then creates a bounded remediation Objective or escalates.

What the user is trying to do: Understand services and investigate incidents.
What must be visible: Service/owner/environment/readiness/incident/change; observation time; live/history distinction; omissions; remediation maturity; verification/write-back.
What the user can do: Open service, inspect evidence/runbook, investigate, create remediation Objective, escalate, or reconcile incident write-back.
When something goes wrong: Live unavailable means current unknown; P0/P1 does not expose executable production rollback.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ08

EN-T-09 - Critical Settings connector review

In plain English: This screen explains an integration's permissions, health, data use, and lifecycle action in ordinary language. On tablet, the same meaning must remain clear.

Why this screen exists: Connecting, disconnecting, or deleting data can have broad consequences and must be reviewed explicitly.
Example: Before authorising Slack, the admin sees the requested access; if authorisation fails, no channels are read.

What the user is trying to do: Review connector capability and lifecycle.
What must be visible: Provider name; account; scopes/capabilities; connection/permission/availability/freshness/capability dimensions; retention/legal hold; export/purge effect.
What the user can do: Authorise/validate, narrow/rotate, reconnect, export, disconnect, queue/verify purge, or cancel.
When something goes wrong: OAuth denial, drift, rate limit, failed write, partial purge, and legal hold have separate recovery; secrets remain write-only.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ09

EN-T-09A - Deep link authorized

In plain English: This confirms that an incoming link is valid and the user is allowed to open the exact Enterprise item. On tablet, the same meaning must remain clear.

Why this screen exists: Protected details appear only after identity, product view, permission, expiry, and source checks pass.
Example: A Jira link opens the intended Objective and action after authorization.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ12

EN-T-09D - Deep link denied

In plain English: This says the user is not allowed to open the item referenced by an incoming link, without exposing what the protected item contains. On tablet, the same meaning must remain clear.

Why this screen exists: A denied link must not leak the company, ticket, Objective, source, or action through titles, previews, or accessibility text.
Example: The user can request access or return safely, but cannot see the protected target.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 834x960 (tablet); 2 canonical actions; 2 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ12

EN-T-09E - Deep link expired

In plain English: This explains that the incoming link is no longer valid and offers a safe reissue or return path. On tablet, the same meaning must remain clear.

Why this screen exists: Expired links cannot restore old permission or disclose the target.
Example: The user requests a fresh link without seeing protected information from the expired one.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 834x960 (tablet); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ12

EN-T-09W - Deep link wrong

In plain English: This says the link belongs to a different product view or account and offers a safe way to switch. On tablet, the same meaning must remain clear.

Why this screen exists: The screen must not reveal the protected target before the correct view is authorised.
Example: A Studio session receives an Enterprise link and is asked to switch without seeing the company ticket title.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 834x960 (tablet); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ12

EN-T-10 - Connector authorization failed recovery

In plain English: This screen explains an integration's permissions, health, data use, and lifecycle action in ordinary language. On tablet, the same meaning must remain clear.

Why this screen exists: Connecting, disconnecting, or deleting data can have broad consequences and must be reviewed explicitly.
Example: Before authorising Slack, the admin sees the requested access; if authorisation fails, no channels are read.

What the user is trying to do: Review connector capability and lifecycle.
What must be visible: Provider name; account; scopes/capabilities; connection/permission/availability/freshness/capability dimensions; retention/legal hold; export/purge effect.
What the user can do: Authorise/validate, narrow/rotate, reconnect, export, disconnect, queue/verify purge, or cancel.
When something goes wrong: OAuth denial, drift, rate limit, failed write, partial purge, and legal hold have separate recovery; secrets remain write-only.

Design reference: 834x960 (tablet); 4 canonical actions; 4 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ09

EN-T-10C - Continuity token created

In plain English: This confirms that a one-time cross-device handoff has been created. On tablet, the same meaning must remain clear.

Why this screen exists: The link contains only a short-lived reference, not the protected Objective or evidence itself.
Example: A mobile user creates a desktop handoff that expires soon and can be revoked.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ14

EN-T-10D - Continuity completed

In plain English: This confirms that the handoff opened successfully and the one-time token has been consumed. On tablet, the same meaning must remain clear.

Why this screen exists: A used token cannot be replayed.
Example: The desktop restores the authorised Objective once, then invalidates the link.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 834x960 (tablet); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ14

EN-T-10O - Continuity opened

In plain English: This is the checking stage after a handoff link is opened; it does not restore protected context yet. On tablet, the same meaning must remain clear.

Why this screen exists: The destination must re-check the user, permission, and source version.
Example: The desktop verifies the same user and current source before showing the Objective.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ14

EN-T-10R - Continuity revoked

In plain English: This confirms that the user or system cancelled the handoff and it can no longer be used. On tablet, the same meaning must remain clear.

Why this screen exists: Revocation must be immediate and visible.
Example: A user revokes a lost-device handoff and the old link becomes unusable.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ14

EN-T-10S - Continuity source changed

In plain English: This blocks restoration because the underlying source changed after the handoff was created. On tablet, the same meaning must remain clear.

Why this screen exists: The user must not unknowingly act on stale code, tickets, or evidence.
Example: The repository SHA changed, so the user reviews the difference and creates fresh authority.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ14

EN-T-10X - Continuity expired

In plain English: This says the cross-device handoff expired before use. On tablet, the same meaning must remain clear.

Why this screen exists: An expired token must not restore the old context.
Example: The user returns to mobile and creates a new handoff if the work is still needed.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ14

EN-T-11 - Inspect drawer open

In plain English: This is the temporary read-only view for checking source evidence or details while preserving the main task. On tablet, the same meaning must remain clear.

Why this screen exists: Looking at context must not silently edit the Objective or mix search history with the main conversation.
Example: A reviewer opens a citation, reads it, and closes Inspect back to the same approval control.

What the user is trying to do: Complete the named Enterprise job.
What must be visible: Organisation/team/Objective identity; state tuple; scope; authority; evidence/freshness; maturity; effect; recovery; audit linkage.
What the user can do: Take the strongest valid next action or return without losing Objective state.
When something goes wrong: Loading, empty, partial, denied, unavailable, failed, recovered, cancelled, and superseded states remain honest and recoverable.

Design reference: 834x960 (tablet); 4 canonical actions; 1 wired; 3 explicitly inert.; 1 direct destinations.
Related journeys: EJ13

EN-T-12 - Scope review

In plain English: This is the review screen shown before work starts. It explains what will change, where, how much it may cost, how it will be checked, and how to recover. On tablet, the same meaning must remain clear.

Why this screen exists: Approval must apply to one exact plan and scope, not to vague future activity.
Example: An approver checks the team, repository, branch, environment, cost, and evidence requirements before allowing the plan to proceed.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ03

EN-T-20R - Context authority resolved

In plain English: This confirms that an authorised person has chosen which source should govern the work. On tablet, the same meaning must remain clear.

Why this screen exists: The decision becomes part of the history and can be replaced later if the source changes.
Example: The team chooses the repository contract over an older document, and planning continues using that version.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 834x960 (tablet); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ03

Mobile screens

EN-M-00 - Navigation overlay

In plain English: This is the phone navigation menu for moving between Home, Objectives, Intelligence, Services, Evidence, and Settings. On phone, the same meaning must remain clear.

Why this screen exists: Mobile has too little space for permanent side panes, so navigation must remain clear and reversible.
Example: The user opens the menu, chooses Objectives, and returns without losing draft work.

What the user is trying to do: Complete the named Enterprise job.
What must be visible: Organisation/team/Objective identity; state tuple; scope; authority; evidence/freshness; maturity; effect; recovery; audit linkage.
What the user can do: Take the strongest valid next action or return without losing Objective state.
When something goes wrong: Loading, empty, partial, denied, unavailable, failed, recovered, cancelled, and superseded states remain honest and recoverable.

Design reference: 390x844 (mobile); 0 canonical actions; 0 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ14

EN-M-00A - Neutral view choice

In plain English: This is the safe front door where the user chooses personal Studio or company Enterprise before any protected information appears. On phone, the same meaning must remain clear.

Why this screen exists: It prevents personal and company context from being mixed.
Example: A user selects Enterprise, authenticates for the company, and then sees only company data they may access.

What the user is trying to do: Choose the correct product boundary.
What must be visible: Neutral Studio/Enterprise choices; no tenant/project/Objective/source/member/memory disclosure; target authorisation and separate URL.
What the user can do: Choose a product or return safely.
When something goes wrong: Wrong account/view or denied target clears protected context and returns to neutral choice.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 5 direct destinations.
Related journeys: EJ11

EN-M-01 - Home needs you

In plain English: This is the user's Enterprise inbox. It highlights Objectives that need a decision, access, approval, or recovery action. On phone, the same meaning must remain clear.

Why this screen exists: People should immediately know what needs them, rather than search through a generic dashboard.
Example: A manager sees one plan awaiting approval and one blocked Objective, then opens the most urgent item.

What the user is trying to do: Find the next Objective decision.
What must be visible: Organisation/team; Needs you; ready/running/blocked Objective rows; source/approval/access causes; filters; maturity; proposal-only create/link.
What the user can do: Open/filter work, create/link Objective, reconnect source, or resolve required authority.
When something goes wrong: Empty, source unavailable, permission-limited, loading, and no-ready-work remain distinct.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ02

EN-M-01F - Objective filters

In plain English: This lets the user narrow the Objective list by team, owner, status, urgency, or other useful criteria. On phone, the same meaning must remain clear.

Why this screen exists: A large company may have hundreds of active Objectives, so people need to find the work relevant to them without changing or hiding the underlying records.
Example: A manager filters to blocked Objectives owned by the payments team, then clears the filter to return to the full list.

What the user is trying to do: Find the next Objective decision.
What must be visible: Organisation/team; Needs you; ready/running/blocked Objective rows; source/approval/access causes; filters; maturity; proposal-only create/link.
What the user can do: Open/filter work, create/link Objective, reconnect source, or resolve required authority.
When something goes wrong: Empty, source unavailable, permission-limited, loading, and no-ready-work remain distinct.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 5 direct destinations.
Related journeys: EJ02

EN-M-02 - Objective context

In plain English: This is the evidence-and-scope view for one Objective. It shows what information the product is using and where that information came from. On phone, the same meaning must remain clear.

Why this screen exists: A user should be able to judge whether the current plan is based on the right repository, tickets, documents, discussions, and source versions.
Example: Before approving a plan, a lead checks that the correct Jira ticket, repository branch, and current architecture document are included.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ03

EN-M-02I - Inspect source evidence

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On phone, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 390x844 (mobile); 0 canonical actions; 0 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ13

EN-M-02Z - Zero-info setup brief

In plain English: This is the honest first-use experience for a company that has not connected any source or created any Objective yet. On phone, the same meaning must remain clear.

Why this screen exists: The product must guide setup without pretending that sample teams, tickets, or history are real.
Example: The admin can describe the first goal, review a setup proposal, or connect a source such as Jira or GitHub.

What the user is trying to do: Bootstrap a truthful empty tenant.
What must be visible: Empty-state explanation; organisation/team/setup intent; source options; proposal-only Brief; permissions/effect review; cancel/resume.
What the user can do: Open scoped Ask, draft setup, review proposal, or connect a source.
When something goes wrong: No sample data may look real; failed setup preserves a safe empty tenant and resumable draft.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ01

EN-M-03 - Plan approval

In plain English: This is the review screen shown before work starts. It explains what will change, where, how much it may cost, how it will be checked, and how to recover. On phone, the same meaning must remain clear.

Why this screen exists: Approval must apply to one exact plan and scope, not to vague future activity.
Example: An approver checks the team, repository, branch, environment, cost, and evidence requirements before allowing the plan to proceed.

What the user is trying to do: Review bounded Plan authority.
What must be visible: Plan/version; tasks/tests/dependencies; target/SHA/environment; actor/approver; cost/time/resource caps; expiry; evidence; effect; recovery.
What the user can do: Approve, reject, change scope, inspect proof, revoke, or re-review.
When something goes wrong: Rejected/expired/invalidated states cannot execute; material changes require fresh approval.

Design reference: 390x844 (mobile); 4 canonical actions; 4 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ04

EN-M-04 - Working Objective

In plain English: This is the live progress view for an Objective that is currently being worked on. On phone, the same meaning must remain clear.

Why this screen exists: Users need meaningful phases, proof, budget, and pause/cancel controls - not a stream of internal agent logs.
Example: A developer sees the current task, files affected, test state, remaining budget, and saved checkpoint.

What the user is trying to do: Track current Objective execution.
What must be visible: Semantic phase; route/provider; source SHA; budget; durable checkpoint; live verification; blockers; partial/superseded attempts; pause/cancel/recovery.
What the user can do: Inspect, pause, cancel-preserve, request access, retry, re-plan, or open successor.
When something goes wrong: Process/outcome stay separate; recovery resumes exact checkpoint and superseded work remains evidence.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ05

EN-M-04B - Working blocked recovery

In plain English: This shows that work cannot currently continue, why it stopped, and the last safe checkpoint. On phone, the same meaning must remain clear.

Why this screen exists: A blocked process is not a failed product outcome, and users need a safe next move.
Example: Tests cannot reach the preview environment; the user requests access or pauses without losing completed work.

What the user is trying to do: Track current Objective execution.
What must be visible: Semantic phase; route/provider; source SHA; budget; durable checkpoint; live verification; blockers; partial/superseded attempts; pause/cancel/recovery.
What the user can do: Inspect, pause, cancel-preserve, request access, retry, re-plan, or open successor.
When something goes wrong: Process/outcome stay separate; recovery resumes exact checkpoint and superseded work remains evidence.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ05

EN-M-05 - Evidence and delivery

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On phone, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 5 direct destinations.
Related journeys: EJ06

EN-M-05A - Scoped Ask

In plain English: This lets the user ask a question while explicitly choosing which team, Objective, and sources may be searched. On phone, the same meaning must remain clear.

Why this screen exists: Users need to understand the boundary of the answer before trusting it.
Example: A user asks who owns a service and limits the search to the current Objective and its approved sources.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ07

EN-M-05P - Partial write-back recovery

In plain English: This screen separates proof of the work from delivery to other systems and shows the result of each external update independently. On phone, the same meaning must remain clear.

Why this screen exists: One successful update must not hide another failed or unknown update.
Example: GitHub and Jira update successfully while Confluence fails; only the failed Confluence action is eligible for retry.

What the user is trying to do: Inspect proof and external delivery.
What must be visible: Verification plan/run/outcome; deterministic versus inferred/unmeasured evidence; source/environment; delivery target; external effect; correlation/idempotency; rollback/reconciliation.
What the user can do: Open proof, run missing verification, review delivery, retry valid subset, reconcile, or hand off.
When something goes wrong: Failed/inconclusive/partial/unmeasured and unknown remote effects remain explicit; never blind-retry external writes.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 5 direct destinations.
Related journeys: EJ14

EN-M-06 - Intelligence scoped partial

In plain English: This is the question-and-answer area for approved company context, including honest states when sources are missing, conflicting, or unreachable. On phone, the same meaning must remain clear.

Why this screen exists: The answer should be useful without pretending to know more than the evidence supports.
Example: A user asks about planned work and receives citations plus a clear note that one source is unavailable.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ07

EN-M-06D - Intelligence permission denied

In plain English: This is the question-and-answer area for approved company context, including honest states when sources are missing, conflicting, or unreachable. On phone, the same meaning must remain clear.

Why this screen exists: The answer should be useful without pretending to know more than the evidence supports.
Example: A user asks about planned work and receives citations plus a clear note that one source is unavailable.

What the user is trying to do: Ask a permission-scoped read-only question.
What must be visible: Scope; permitted sources; retrieval state; cited answer; omissions/conflicts/freshness; denied/unavailable/no-result distinction; proposed follow-up.
What the user can do: Submit/refine query, inspect citations, compare conflict, request access, refresh, or propose a bounded Objective/action.
When something goes wrong: Denied reveals no protected claims; organisation-wide Intelligence is P1 and explicitly labelled.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ07

EN-M-07 - Incident conditional remediation

In plain English: This is the investigation workspace for an active service incident. On phone, the same meaning must remain clear.

Why this screen exists: Teams need current evidence, owners, impact, and safe next actions in one place.
Example: An SRE reviews live and historical signals, then creates a bounded remediation Objective or escalates.

What the user is trying to do: Understand services and investigate incidents.
What must be visible: Service/owner/environment/readiness/incident/change; observation time; live/history distinction; omissions; remediation maturity; verification/write-back.
What the user can do: Open service, inspect evidence/runbook, investigate, create remediation Objective, escalate, or reconcile incident write-back.
When something goes wrong: Live unavailable means current unknown; P0/P1 does not expose executable production rollback.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ08

EN-M-08 - Critical Settings connector review

In plain English: This screen explains an integration's permissions, health, data use, and lifecycle action in ordinary language. On phone, the same meaning must remain clear.

Why this screen exists: Connecting, disconnecting, or deleting data can have broad consequences and must be reviewed explicitly.
Example: Before authorising Slack, the admin sees the requested access; if authorisation fails, no channels are read.

What the user is trying to do: Review connector capability and lifecycle.
What must be visible: Provider name; account; scopes/capabilities; connection/permission/availability/freshness/capability dimensions; retention/legal hold; export/purge effect.
What the user can do: Authorise/validate, narrow/rotate, reconnect, export, disconnect, queue/verify purge, or cancel.
When something goes wrong: OAuth denial, drift, rate limit, failed write, partial purge, and legal hold have separate recovery; secrets remain write-only.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 7 direct destinations.
Related journeys: EJ09

EN-M-08F - Connector authorization failed

In plain English: This screen explains an integration's permissions, health, data use, and lifecycle action in ordinary language. On phone, the same meaning must remain clear.

Why this screen exists: Connecting, disconnecting, or deleting data can have broad consequences and must be reviewed explicitly.
Example: Before authorising Slack, the admin sees the requested access; if authorisation fails, no channels are read.

What the user is trying to do: Review connector capability and lifecycle.
What must be visible: Provider name; account; scopes/capabilities; connection/permission/availability/freshness/capability dimensions; retention/legal hold; export/purge effect.
What the user can do: Authorise/validate, narrow/rotate, reconnect, export, disconnect, queue/verify purge, or cancel.
When something goes wrong: OAuth denial, drift, rate limit, failed write, partial purge, and legal hold have separate recovery; secrets remain write-only.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ09

EN-M-09A - Deep link authorized

In plain English: This confirms that an incoming link is valid and the user is allowed to open the exact Enterprise item. On phone, the same meaning must remain clear.

Why this screen exists: Protected details appear only after identity, product view, permission, expiry, and source checks pass.
Example: A Jira link opens the intended Objective and action after authorization.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 3 direct destinations.
Related journeys: EJ12

EN-M-09D - Deep link denied

In plain English: This says the user is not allowed to open the item referenced by an incoming link, without exposing what the protected item contains. On phone, the same meaning must remain clear.

Why this screen exists: A denied link must not leak the company, ticket, Objective, source, or action through titles, previews, or accessibility text.
Example: The user can request access or return safely, but cannot see the protected target.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 1 direct destinations.
Related journeys: EJ12

EN-M-09E - Deep link expired

In plain English: This explains that the incoming link is no longer valid and offers a safe reissue or return path. On phone, the same meaning must remain clear.

Why this screen exists: Expired links cannot restore old permission or disclose the target.
Example: The user requests a fresh link without seeing protected information from the expired one.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ12

EN-M-09W - Deep link wrong

In plain English: This says the link belongs to a different product view or account and offers a safe way to switch. On phone, the same meaning must remain clear.

Why this screen exists: The screen must not reveal the protected target before the correct view is authorised.
Example: A Studio session receives an Enterprise link and is asked to switch without seeing the company ticket title.

What the user is trying to do: Resolve an external Enterprise link.
What must be visible: Origin and intended object/action class; account/view; permission; expiry; source version; authorised/redacted result; request-access/reissue/return-safe path.
What the user can do: Authenticate, switch view/account, open exact authorised target, request access, reissue, or return.
When something goes wrong: Only authorised states reveal protected identifiers; wrong/expired/denied/revoked/source-changed remain distinct.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 2 direct destinations.
Related journeys: EJ12

EN-M-10C - Secure handoff created

In plain English: This confirms that a one-time cross-device handoff has been created. On phone, the same meaning must remain clear.

Why this screen exists: The link contains only a short-lived reference, not the protected Objective or evidence itself.
Example: A mobile user creates a desktop handoff that expires soon and can be revoked.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ14

EN-M-10D - Continuity completed

In plain English: This confirms that the handoff opened successfully and the one-time token has been consumed. On phone, the same meaning must remain clear.

Why this screen exists: A used token cannot be replayed.
Example: The desktop restores the authorised Objective once, then invalidates the link.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 390x844 (mobile); 1 canonical actions; 1 wired; 0 explicitly inert.; 4 direct destinations.
Related journeys: EJ14

EN-M-10O - Handoff opened

In plain English: This is the checking stage after a handoff link is opened; it does not restore protected context yet. On phone, the same meaning must remain clear.

Why this screen exists: The destination must re-check the user, permission, and source version.
Example: The desktop verifies the same user and current source before showing the Objective.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 7 direct destinations.
Related journeys: EJ14

EN-M-10R - Handoff revoked

In plain English: This confirms that the user or system cancelled the handoff and it can no longer be used. On phone, the same meaning must remain clear.

Why this screen exists: Revocation must be immediate and visible.
Example: A user revokes a lost-device handoff and the old link becomes unusable.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ14

EN-M-10S - Handoff source changed

In plain English: This blocks restoration because the underlying source changed after the handoff was created. On phone, the same meaning must remain clear.

Why this screen exists: The user must not unknowingly act on stale code, tickets, or evidence.
Example: The repository SHA changed, so the user reviews the difference and creates fresh authority.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 390x844 (mobile); 3 canonical actions; 3 wired; 0 explicitly inert.; 7 direct destinations.
Related journeys: EJ14

EN-M-10X - Handoff expired

In plain English: This says the cross-device handoff expired before use. On phone, the same meaning must remain clear.

Why this screen exists: An expired token must not restore the old context.
Example: The user returns to mobile and creates a new handoff if the work is still needed.

What the user is trying to do: Continue securely across devices.
What must be visible: One-time same-user token; scoped Objective/evidence; expiry/revocation; destination permission; source-version check; open/complete receipt; origin return path.
What the user can do: Create/copy/open/revoke/reissue/complete handoff.
When something goes wrong: Opening is not restoration; expired/revoked/source-changed/denied states never expose protected context.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 5 direct destinations.
Related journeys: EJ14

EN-M-12 - Scope review

In plain English: This is the review screen shown before work starts. It explains what will change, where, how much it may cost, how it will be checked, and how to recover. On phone, the same meaning must remain clear.

Why this screen exists: Approval must apply to one exact plan and scope, not to vague future activity.
Example: An approver checks the team, repository, branch, environment, cost, and evidence requirements before allowing the plan to proceed.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 5 direct destinations.
Related journeys: EJ03

EN-M-20R - Context authority resolved

In plain English: This confirms that an authorised person has chosen which source should govern the work. On phone, the same meaning must remain clear.

Why this screen exists: The decision becomes part of the history and can be replaced later if the source changes.
Example: The team chooses the repository contract over an older document, and planning continues using that version.

What the user is trying to do: Review scoped source context.
What must be visible: Exact Objective/source scope; citations; permission/availability/freshness/epistemic state; omissions/conflicts; governing authority; affected plan/decision.
What the user can do: Inspect/compare source, refresh, change scope, request access, choose governing source, or continue.
When something goes wrong: Denied redacts protected data; unavailable differs from no result; source change invalidates dependent authority.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ03

EN-M-55M - Route unavailable usage limit

In plain English: This screen explains which AI provider and model will be used, whether it is available, what it may cost, and which alternatives are allowed. On phone, the same meaning must remain clear.

Why this screen exists: The system must not silently switch providers, change data handling, or spend more money.
Example: The selected model reaches its limit, so the user reviews an approved alternative before resuming.

What the user is trying to do: Govern provider/model routing.
What must be visible: Provider/model names; key source; health; policy/maturity; data/budget boundary; usage/limit; eligible alternatives; approval and route-validation receipt.
What the user can do: Select/review allowed route, request policy change, validate, resume, or remain blocked.
When something goes wrong: No silent failover/substitution/spend; DeepSeek/OpenRouter/Fireworks remain named without logos.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 6 direct destinations.
Related journeys: EJ10

EN-M-55R - Model route review

In plain English: This screen explains which AI provider and model will be used, whether it is available, what it may cost, and which alternatives are allowed. On phone, the same meaning must remain clear.

Why this screen exists: The system must not silently switch providers, change data handling, or spend more money.
Example: The selected model reaches its limit, so the user reviews an approved alternative before resuming.

What the user is trying to do: Govern provider/model routing.
What must be visible: Provider/model names; key source; health; policy/maturity; data/budget boundary; usage/limit; eligible alternatives; approval and route-validation receipt.
What the user can do: Select/review allowed route, request policy change, validate, resume, or remain blocked.
When something goes wrong: No silent failover/substitution/spend; DeepSeek/OpenRouter/Fireworks remain named without logos.

Design reference: 390x844 (mobile); 2 canonical actions; 2 wired; 0 explicitly inert.; 7 direct destinations.
Related journeys: EJ10

Responsive and accessibility meaning

Desktop may show navigation, the main work, and Inspect together. Tablet keeps the main task visible and opens Inspect as a large temporary panel. Mobile shows one main surface at a time and uses dedicated routes for details and secure desktop handoff. The meaning, permission, evidence, recovery, and current decision must remain the same on every device.

Accessibility in ordinary language

Every action must be reachable without a mouse, focus must be visible, dialogs must keep and return focus correctly, status changes must be announced, text must survive zoom and translation, touch targets must be large enough, and colour must never be the only way a person understands a state. Protected information must also be absent from screen-reader output when access is denied.

What remains outside this document

The Figma designs and this PRD explain intended experience. Engineering must still prove real permissions, redaction, connector behavior, retries, storage, browser accessibility, device behavior, and the 109 formal acceptance requirements. Coding remains paused until the product owner explicitly approves the design.

Open the canonical Enterprise Figma file